Worker-Aware Warm-Started Tabu Search for Flexible Job Shop Scheduling with Worker Flexibility

Anonymous Author(s)

ABSTRACT

The flexible job shop scheduling problem with worker flexibility (FJSSP-W) extends the classical FJSSP by assigning each operation to both an eligible machine and a compatible worker. Processing times depend on the chosen operation, machine, and worker. This paper describes a worker-aware tabu search workflow for FJSSP-W. The method follows the single-solution local-search philosophy of adaptive-weighting local search for FJSSP, but augments the representation, feasibility checks, disjunctive graph, neighborhood screening, and promotion protocol to account for worker-resource conflicts. Starting from high-quality feasible schedules, the solver alternates bounded warm-start intensification, approximate neighborhood screening, exact validation, and submission-grade regeneration with ten distinct random seeds per instance. On the 30 Scenario 1 instances used in our current evaluation, the latest audited artifact improves the reference best makespan on 11 instances, matches it on the remaining 19, and has no best-score regressions. The summed best makespan is reduced from 38,496 to 38,425, and the summed mean makespan over ten runs per instance is reduced from 39,549.3 to 38,464.3. These results suggest that carefully adapting trajectory local search to the worker dimension can be competitive even without a heavy population model.

CCS CONCEPTS

• **Computing methodologies** → **Search methodologies**; • **Applied computing** → **Operations research**.

KEYWORDS

Flexible job shop scheduling, worker flexibility, tabu search, local search, competition, warm start

ACM Reference Format:
Anonymous Author(s). 2026. Worker-Aware Warm-Started Tabu Search for Flexible Job Shop Scheduling with Worker Flexibility. In *Proceedings of Genetic and Evolutionary Computation Conference (GECCO Draft)*. ACM, New York, NY, USA, 5 pages.

1 INTRODUCTION

Flexible job shop scheduling (FJSSP) is a canonical production-scheduling problem in which each operation may be processed on one of several eligible machines. Recent benchmark work extends this setting to FJSSP with worker flexibility (FJSSP-W), where worker availability and worker-dependent processing times must be considered together with machines [1]. This additional resource dimension changes both the feasible schedule space and the local-search landscape: a move that is harmless in FJSSP can become infeasible or strongly misleading once workers are included.

Our starting point is the adaptive-weighting local search (AWLS) algorithm of Zhang et al. [2]: a strong single-solution trajectory search can be highly competitive for FJSSP when tabu memory, critical-path neighborhoods, approximate move evaluation, and adaptive operation weights are combined. Rather than presenting a population-based evolutionary solver, this paper documents a pragmatic FJSSP-W competition workflow built around worker-aware tabu search, warm starts, and strict promotion gates. The contributions are:

- a worker-aware encoding and disjunctive-graph view that adds worker arcs alongside job-precedence and machine arcs;
- a tabu-search neighborhood that includes operation-order, machine-assignment, worker-assignment, and joint assignment moves;
- a bounded exploration and regeneration workflow that keeps exploratory best rows separate from ten-seed submission artifacts; and
- an audited result set on 30 Scenario 1 instances, improving the reference best makespan on 11 instances without any best-score regression.

2 PROBLEM DEFINITION

Let O be the set of operations. Each operation $i \in O$ belongs to a job and therefore has fixed precedence constraints with the other operations of the same job. For each operation, a feasible processing option is a pair (m, w) consisting of an eligible machine m and an eligible worker w. The processing time is $p_{i,m,w}$; infeasible triples have no processing time. A solution assigns each operation a start time s_i, a machine m_i, and a worker w_i. The schedule must satisfy:

(1) job precedence constraints;
(2) non-overlap constraints for operations assigned to the same machine;
(3) non-overlap constraints for operations assigned to the same worker; and
(4) eligibility constraints $p_{i,m_i,w_i} > 0$.

The objective is to minimize the makespan

$$C_{\max} = \max_{i \in O} \left(s_i + p_{i,m_i,w_i} \right).$$

3 METHOD

3.1 From AWLS to FJSSP-W

The original AWLS is designed for the classical FJSSP, where each operation is assigned to a machine and inserted into a position on that machine. It maintains a single incumbent solution, repeatedly selects a promising move from critical-path neighborhoods, forbids recent move attributes by a tabu list, and uses a fast estimated makespan to avoid exact decoding of every neighbor. Its distinctive component is adaptive weighting: each operation has a cumulative weight and an idle count, and the estimator treats the induced penalty as additional processing time. When a move fails to improve the incumbent, the moved operation becomes less attractive in later estimates; when a new best solution is found, the weights

are reset. This gives AWLS a long-term diversification mechanism complementary to tabu tenure.

FJSSP-W changes the object being moved. A move can no longer be described only as "operation i goes to machine m at position k". It must also specify a compatible worker and a feasible worker-queue position, and its effect must be evaluated on both machine and worker resource graphs. Our solver therefore keeps the AWLS single-solution and tabu-search structure, but generalizes the state, graph, neighborhoods, screening score, and validation step to the worker dimension. The current submission-grade artifact uses this conservative worker-aware variant with exact validation as the acceptance authority; full adaptive operation weighting is retained as a natural next ablation rather than claimed as the source of the reported results.

3.2 Encoding and Decoding

The implementation represents a solution by three vectors:

$$x = (\pi, M, W),$$

where π is a job-sequence representation, M gives one machine assignment for each operation, and W gives the corresponding worker assignment. If job j appears k times in π, the kth appearance denotes the kth operation of job j. Thus job precedence is represented implicitly by the occurrence count, while M_i and W_i select one feasible processing pair for operation i.

The decoder scans π from left to right. When the next operation i of job j is encountered, the decoder schedules it at the earliest time satisfying three ready times:

$$s_i = \max\{R_j^{job},\ R_{M_i}^{mach},\ R_{W_i}^{work}\},$$

where R_j^{job} is the completion time of the previous operation of job j, $R_{M_i}^{mach}$ is the current ready time of the selected machine, and $R_{W_i}^{work}$ is the current ready time of the selected worker. After scheduling i, all three ready times are updated by p_{i,M_i,W_i}. This left-shift decoder produces a feasible active schedule whenever every selected pair (M_i, W_i) is eligible. The representation is compact enough for trajectory search while retaining the two assignment dimensions needed by FJSSP-W.

3.3 Worker-Aware Disjunctive Graph

Classical FJSSP local search often reasons about a disjunctive graph containing conjunctive job arcs and disjunctive machine arcs. For FJSSP-W we use the same idea but add a third arc family for workers. For a decoded schedule we construct:

- job arcs between consecutive operations of each job;
- machine arcs between consecutive operations in each machine queue; and
- worker arcs between consecutive operations in each worker queue.

The worker arcs are not a cosmetic addition. They determine feasibility after worker reassignment moves, affect critical-path estimates, and prevent approximate evaluation from accepting moves that create hidden worker conflicts. In this graph, a critical operation is an operation lying on at least one longest path from the dummy source to the dummy sink. Because a longest path may now pass through either a machine arc or a worker arc, the critical structure can change after a pure worker move even when all machine queues are unchanged.

3.4 Exact Validation

Every accepted move is passed through exact validation. First, the selected machine–worker pair of each touched operation is checked against the precomputed eligibility list. Second, the candidate is decoded with both machine and worker ready times. Third, the decoded schedule is converted to machine and worker queues and checked for overlap. This final check is deliberately redundant: it catches implementation mistakes in approximate screening and prevents an infeasible row from entering a submission artifact.

3.5 Neighborhoods

The search samples a mixed neighborhood. Sequence neighborhoods include swaps, insertions, adjacent swaps, block swaps, block relocations, two-shifts, and inversions. Assignment neighborhoods include machine reassignment, worker reassignment, machine-adjacent and worker-adjacent swaps, and joint machine-worker changes. Operation-specific eligibility lists filter all assignment moves, so infeasible pairs are never promoted to exact evaluation.

The move set is biased toward operations on the current critical structure. For sequence moves, the solver preferentially samples operations that lie near long machine or worker queues, because these queues are more likely to determine $C_{\max}$. For assignment moves, the solver samples alternative pairs from the operation's eligibility list and gives priority to pairs with shorter processing time or lower local queue pressure. The important difference from a standard FJSSP neighborhood is that a reassignment is not a machine-only decision: changing M_i without reconsidering W_i can simply move the bottleneck from a machine queue to a worker queue.

Conceptually, the original AWLS move (i, m, k) is replaced by two families. Resource moves change the selected pair (M_i, W_i), and sequence moves relocate or reorder operations inside a machine queue, a worker queue, or both. For a joint assignment move, the solver first chooses a feasible pair $(m, w) \in E_i$ and then tests insertion positions induced by the current machine and worker queues. This avoids generating a large number of impossible triples and keeps the exact-evaluation budget focused on candidates that can plausibly survive both resource constraints.

3.6 Tabu Search with Approximate Screening

The main search is tabu search with aspiration. Each move type has a tabu key encoding the changed order or assignment attributes. At each iteration the solver samples a neighborhood, applies a fast screening layer, and exactly evaluates only a limited candidate set. The screening layer estimates local effects from duration changes and nearby machine or worker queues. It is deliberately conservative: exact makespan evaluation remains the final authority before a neighbor is accepted or exported. The exact-evaluation cap is important because fully decoding every neighbor is expensive on large instances.

The screening score follows the role played by the AWLS estimator, but with two resource queues instead of one. For a candidate

move a touching operation i, the score combines the processing-time change, local insertion pressure on the target machine, local insertion pressure on the target worker, and an optional AWLS-style operation penalty:

$$\hat{C}(a) = \Delta p_i(a) + \rho_m(a) + \rho_w(a) + \lambda Z_i.$$

Here ρ_m and ρ_w are local queue-pressure estimates, and Z_i denotes the adaptive additional processing time derived from operation weight and idle count in AWLS. In the conservative setting used for the present artifact, $\lambda = 0$; the formula is included to make clear how the original adaptive weighting mechanism plugs into the worker-aware estimator.

Algorithm 1 gives the full search skeleton. The tabu list stores reverse attributes of accepted moves, such as the previous position of an operation or the previous machine–worker pair. A tabu move is ignored unless it satisfies aspiration by improving the best makespan seen in the current run. When no non-tabu improving candidate exists, the least damaging admissible candidate is still accepted; this keeps the search moving across plateaus.

3.7 Why Approximation Is Safe

Approximate evaluation is used only to rank candidates before exact decoding. It never writes a solution to the incumbent set by itself. This separation is essential for FJSSP-W: a move may reduce the processing time of one operation but insert that operation into a congested worker queue, making the true makespan worse. The safe compromise is therefore to let the approximation answer "which candidates deserve exact decoding?" while the exact decoder answers "which candidate is actually feasible and best?". This is stricter than the original FJSSP setting because an estimator that is reasonable on machine queues alone can still miss a newly created worker conflict.

3.8 Warm-Start Promotion Workflow

The competition workflow is as important as the local search itself. We distinguish three artifact levels:

(1) **Exploratory scans**: short bounded runs over selected instances, allowed to replace only one row internally and used only for discovery.
(2) **Submission-grade regeneration**: if an exploratory scan finds a strict improvement, the improved row is used as a warm start and the target instance is regenerated with exactly ten distinct seeds.
(3) **Promotion**: a full 30-instance CSV is promoted only after an audit verifies that every official instance has exactly ten rows and that no best-score regression is introduced.

This protocol avoids a common competition pitfall: mixing one lucky best row into a CSV and treating it as if it satisfied the multi-run submission discipline.

4 EXPERIMENTAL SETUP

4.1 Benchmark and Baseline

Experiments use the FJSSP-W benchmarking environment and Scenario 1 submission format. The candidate artifact is compared against the reference file `atpts_final_submission.csv`. The current audited candidate is the 2026-04-27 `best30_v18` artifact. Both files contain 30 official instances and exactly ten rows per instance.

Algorithm 1 Worker-aware warm-started tabu search

Require: Instance I, seed r, start solution x_0, budget B
Ensure: Best feasible solution $x^\star$

```
Build eligibility lists E_i = {(m, w, p_{i,m,w})} for every operation i
Initialize x ← repair-and-decode(x_0, E) or construct a feasible random solution
x* ← x; initialize tabu list T ← ∅ and AWLS weights Z_i ← 0
while budget B is not exhausted do
    Sample sequence, machine, worker, and joint assignment moves around x
    for all sampled moves a do
        if a selects an ineligible machine–worker pair then
            discard a
        else if a is tabu and does not satisfy aspiration then
            discard a
        else
            score a by duration change, machine pressure, worker pressure, and optional Z_i
        end if
    end for
    Select the best K scored moves for exact validation
    Decode each validated candidate with job, machine, and worker ready times
    Choose the admissible candidate y with the best accepted makespan rule
    if no candidate passes exact validation then
        restart or resample from x*
    else
        x ← y; insert reverse move attributes into T; update optional AWLS weights
        if C_max(x) < C_max(x*) then
            x* ← x
        end if
    end if
end while
return x*
```

4.2 Run Discipline

Submission-grade regeneration uses ten distinct seeds:

$$7, 19, 29, 41, 53, 67, 79, 97, 131, 173.$$

Exploratory scans use small seed batches with per-instance wall-clock timeouts, then regenerate any discovered improvement with the ten submission seeds before promotion. The main promoted second-stage improvements used a cap of 20,000 exact evaluations, 200,000 iterations, and neighborhood size 60 for the regenerated target instance.

Table 1: Audit summary on the 30 Scenario 1 instances.

Category	Count
Strict best improvement	11
Same best, lower average makespan	14
Same best, lower function-evaluation count	5
Best-score regression	0
Official instances	30
Rows per instance	10

4.3 Evaluation Metrics

We report best makespan, average makespan over the ten rows of each instance, and function evaluations to the best row when available. A candidate is classified as:

- **strict better** if its best makespan is smaller than the reference best;
- **same-best average better** if the best makespan is tied but average makespan is lower; and
- **same-best FE better** if best and average makespan are tied but the average number of function evaluations is lower.

5 RESULTS

Table 1 summarizes the 30-instance audit. The candidate improves the reference best makespan on 11 instances, ties the reference best on the other 19, and has no best-score regression.

Table 2 lists the strict best improvements. The largest individual reductions occur on DPpaulli and Hurink vdata instances. The summed reference best makespan over all 30 instances is 38,496; the candidate reduces it to 38,425, a total decrease of 71. The summed ten-run average makespan decreases from 39,549.3 to 38,464.3.

5.1 Second-Stage Discoveries

Two recent improvements illustrate the value of the bounded exploration protocol. First, a bounded scan found a 1069 warm start for `2d_Hurink_vdata_30_workers`; submission-grade regeneration then improved the ten-seed artifact further to 1064. Second, a bounded scan found 1036 for `2d_Hurink_vdata_18_workers`; regeneration made all ten rows equal to 1036 with zero additional function evaluations from that warm start. Both promotions passed the audit gate.

6 DISCUSSION

The results are not merely an artifact of replacing single lucky rows. Every promoted CSV has the full 30-instance shape, exactly ten rows per official instance, and is checked against the reference for regressions. This matters because the strongest practical improvements were often found in exploratory mode before being converted into compliant ten-seed rows.

The method also highlights the importance of worker-aware reasoning. In FJSSP-W, worker queues can be as constraining as machine queues. Neighborhoods that only inspect machine conflicts may accept moves whose apparent gain disappears after worker-resource feasibility is restored. Adding worker arcs and worker-specific adjacency information gives the search a more faithful local model.

There are two limitations. First, this draft reports a competition-engineering workflow rather than a controlled ablation study; the marginal contribution of each neighborhood, the worker-aware screening terms, and the original AWLS adaptive-weighting term has not yet been isolated. Second, the reference comparison uses a 10-row submission file rather than a full independent statistical campaign with 20–30 repetitions per instance. The latter is important for a camera-ready scientific paper.

7 CONCLUSION

We presented a worker-aware, warm-started tabu-search workflow for FJSSP-W. The solver adapts the single-solution local-search philosophy behind AWLS to schedules with both machine and worker conflicts. On the current 30-instance Scenario 1 artifact, the approach improves 11 reference best makespans, ties the remaining 19, and reduces both summed best and summed average makespan with no best-score regression. Future work should re-enable and tune the full AWLS adaptive operation-weight term for FJSSP-W, run a full statistical evaluation, and report ablations for worker arcs, assignment moves, approximate screening, and warm-start promotion.

REFERENCES

[1] David Hutter, Thomas Steinberger, and Michael Hellwig. 2025. Comprehensive Benchmarking Environment for Worker Flexibility in Flexible Job Shop Scheduling Problems. arXiv:2501.16159 [cs.AI] https://arxiv.org/abs/2501.16159

[2] Junjie Zhang, Zhipeng Lü, Junwen Ding, Zhouxing Su, Xinyu Li, and Liang Gao. 2025. An Effective Local Search Algorithm for Flexible Job Shop Scheduling in Intelligent Manufacturing Systems. *Engineering* 50 (2025), 117–127. https://doi.org/10.1016/j.eng.2024.07.022 Available online 2024.

Table 2: Instances where the candidate improves the reference best makespan.

Instance	Ref. best	Cand. best	Delta	Ref. avg.	Cand. avg.
3_DPpaulli_15_workers	2213	2199	-14	2241.1	2208.9
2d_Hurink_vdata_30_workers	1075	1064	-11	1084.3	1068.1
3_DPpaulli_18_workers	2215	2204	-11	2242.0	2210.6
3_DPpaulli_9_workers	2087	2077	-10	2099.2	2083.5
2c_Hurink_rdata_38_workers	1539	1533	-6	1556.8	1536.6
2c_Hurink_rdata_28_workers	789	784	-5	811.9	787.0
2d_Hurink_vdata_18_workers	1041	1036	-5	1047.5	1036.0
6_Fattahi_14_workers	546	543	-3	570.6	544.0
0_BehnkeGeiger_60_workers	403	401	-2	419.6	402.0
3_DPpaulli_1_workers	2482	2480	-2	2528.0	2481.8
6_Fattahi_20_workers	1155	1153	-2	1177.5	1154.8